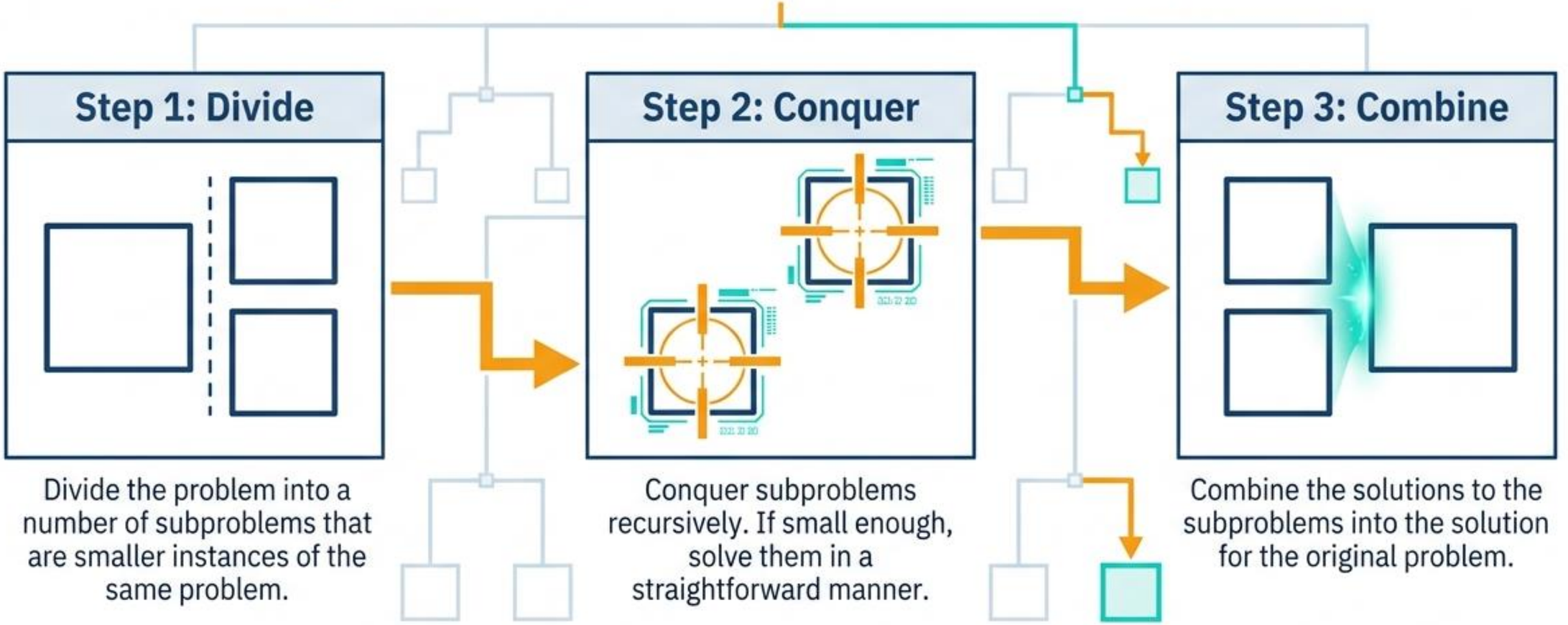
Recursive Algorithms & Complexity Analysis

Formulating and Solving Recurrence Relations in Computer Science



One primary method to implement the Divide and Conquer paradigm is via Recurrence.

Recursion vs. Recurrence

Recursion:
Solving a problem by reducing it to smaller instances of the exact same problem.

Recurrence Relation:
The mathematical formula defining T(n) in terms of its earlier values (e.g., `T(n) = T(n-1) + n`

Essential Properties for Recursive Functions

✓ **1. Base Criteria (Stop Condition)**

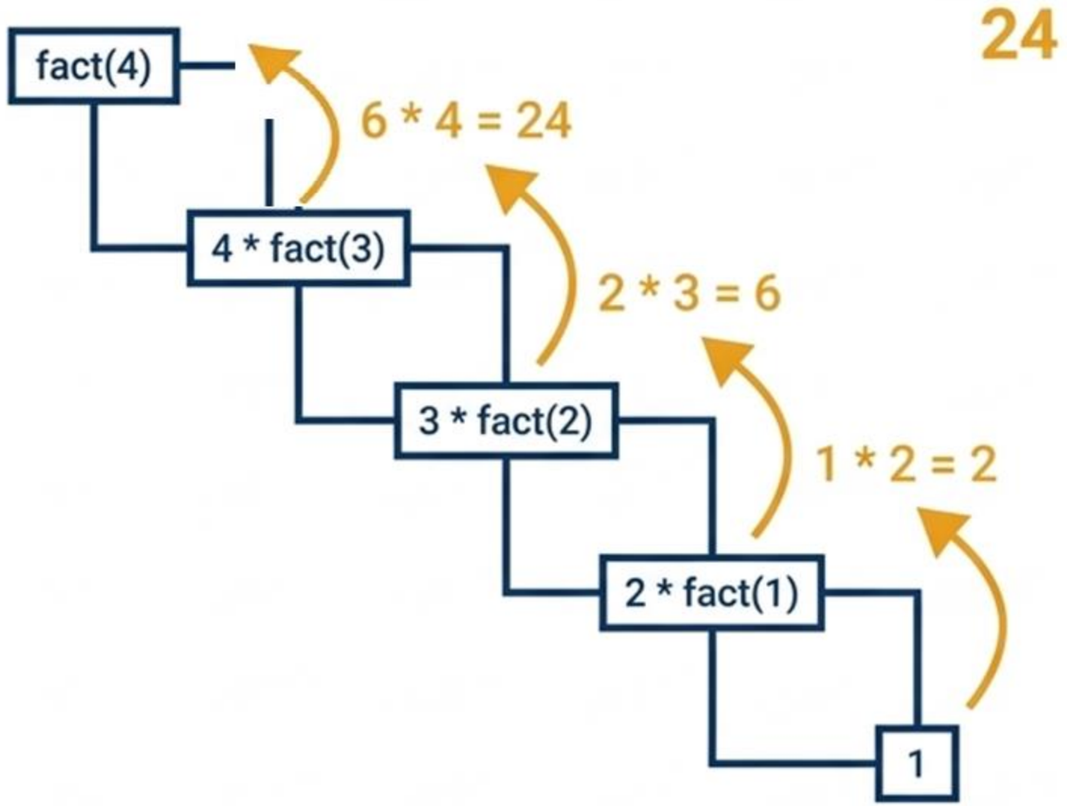
There must be at least one condition where the function solves the problem without calling itself, halting the recursion.

✓ **2. Progressive Approach**

Every recursive call must fundamentally progress or reduce the problem size, bringing it closer to the base criteria.

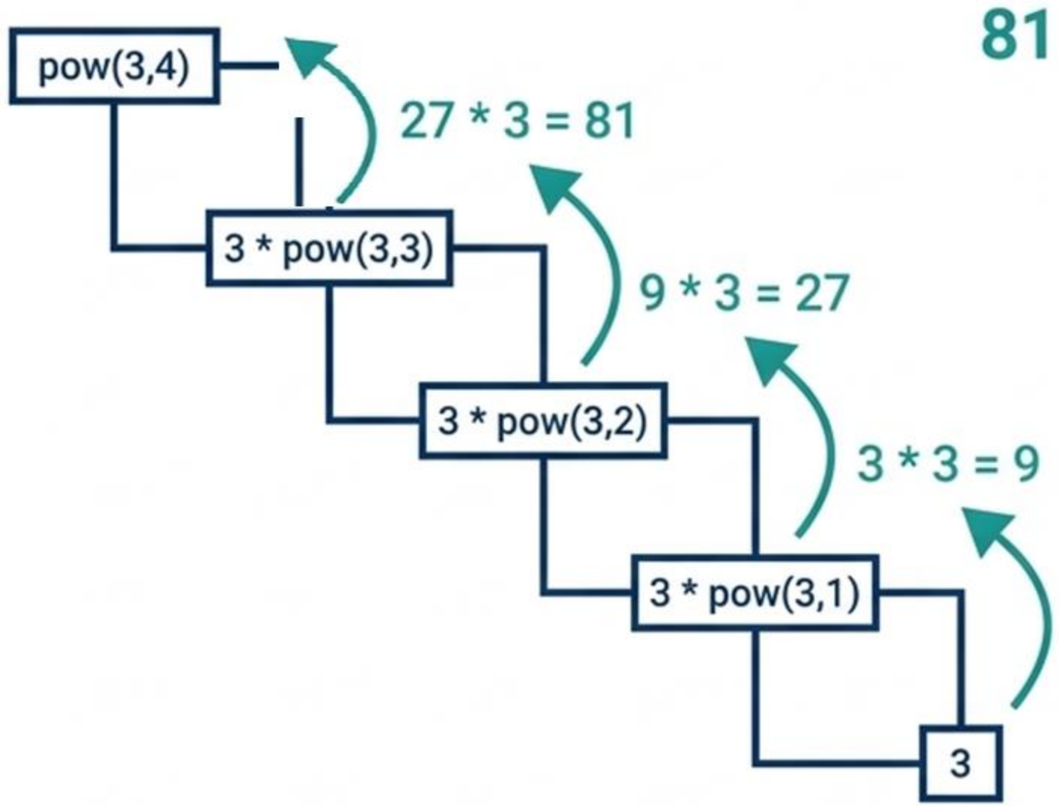
Factorial

$$fact\,(n) = \begin{cases} 1\ if\ n = 1 \\ n \times fact\,(n-1)\ if\ n > 1 \end{cases}$$

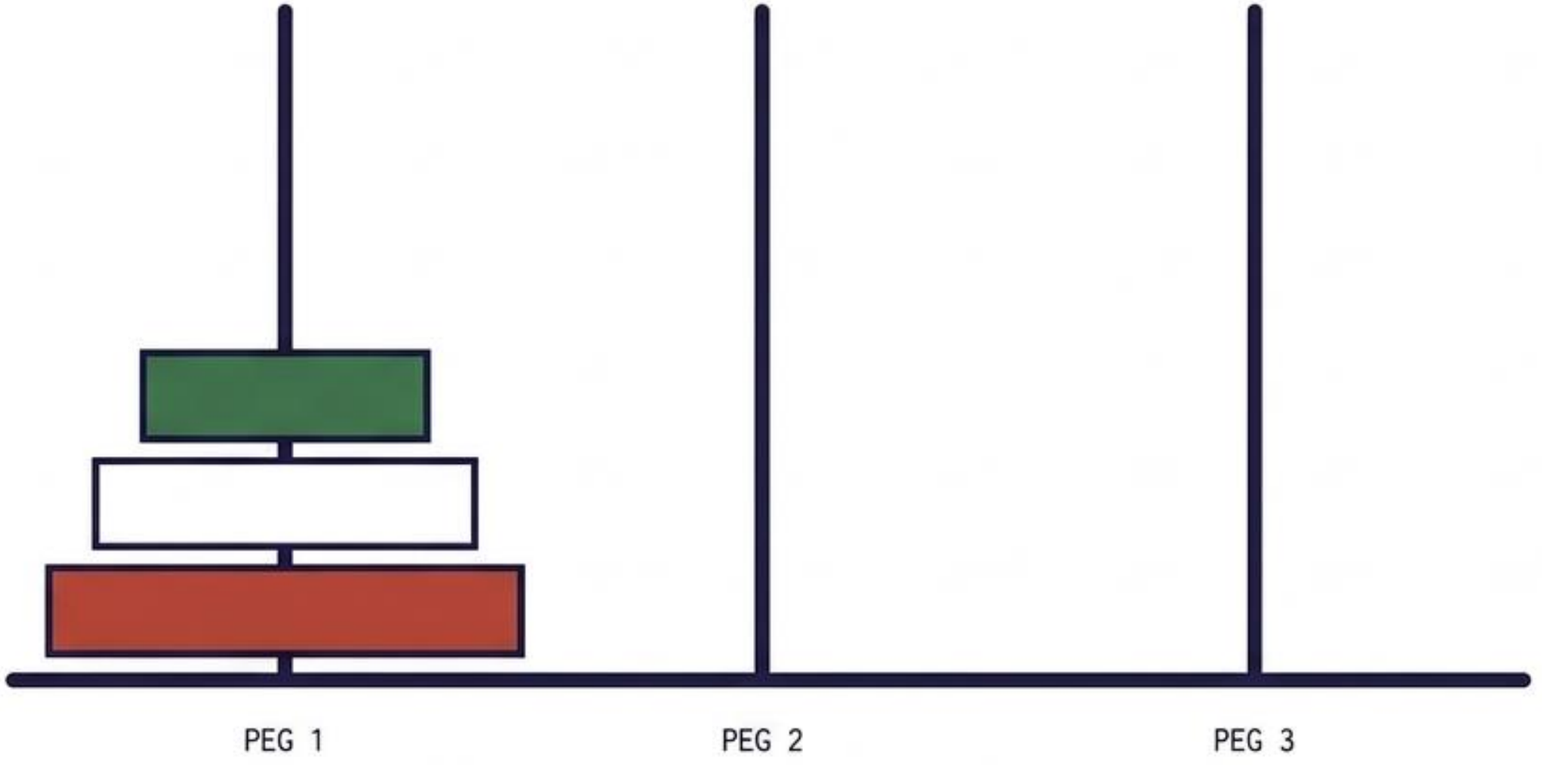


Power

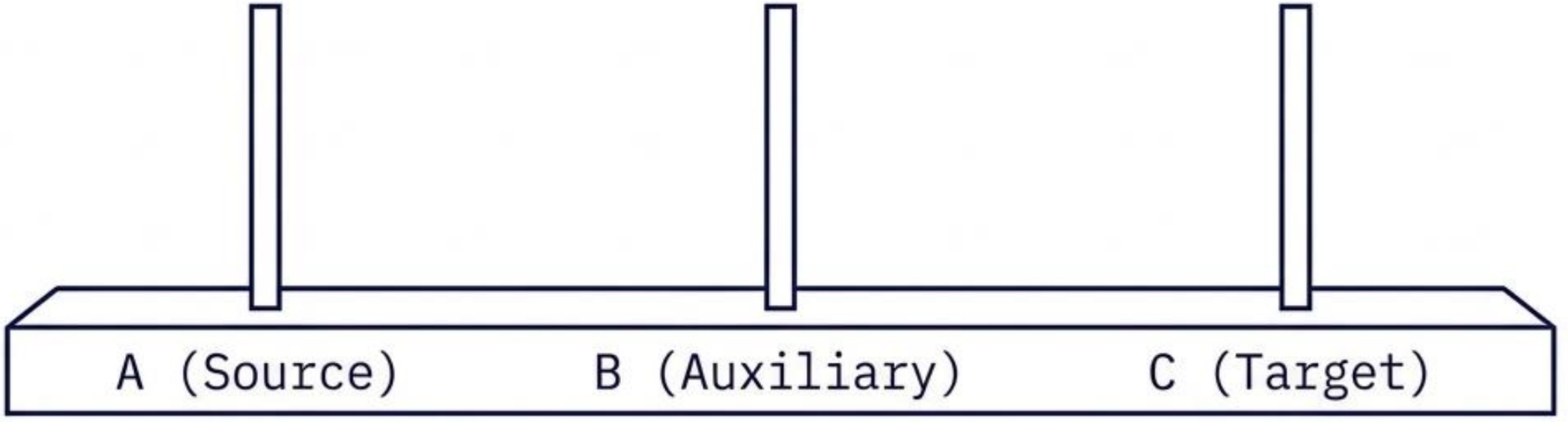
$$pow\,(n,m) = \begin{cases} n\ if\ m = 1 \\ n \times pow(n,m-1) if\ m > 1 \end{cases}$$



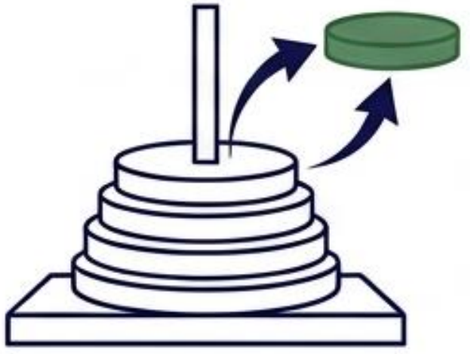
The Tower of Hanoi:
A Visual Guide to Recursion



The Rules of Engagement



Law 1: Atomic Movement



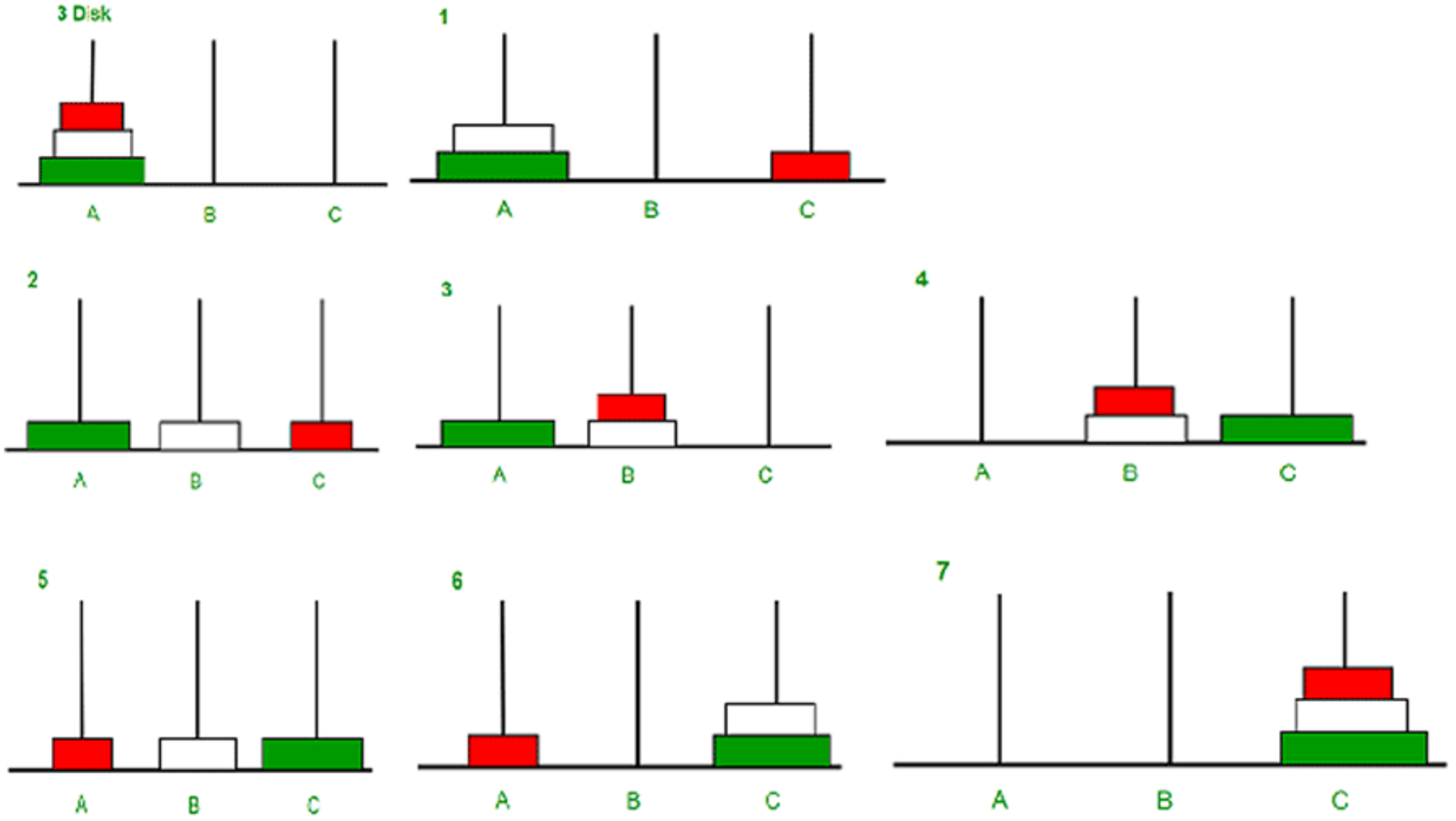
Move only one disk at a time.

Law 2: Structural Integrity

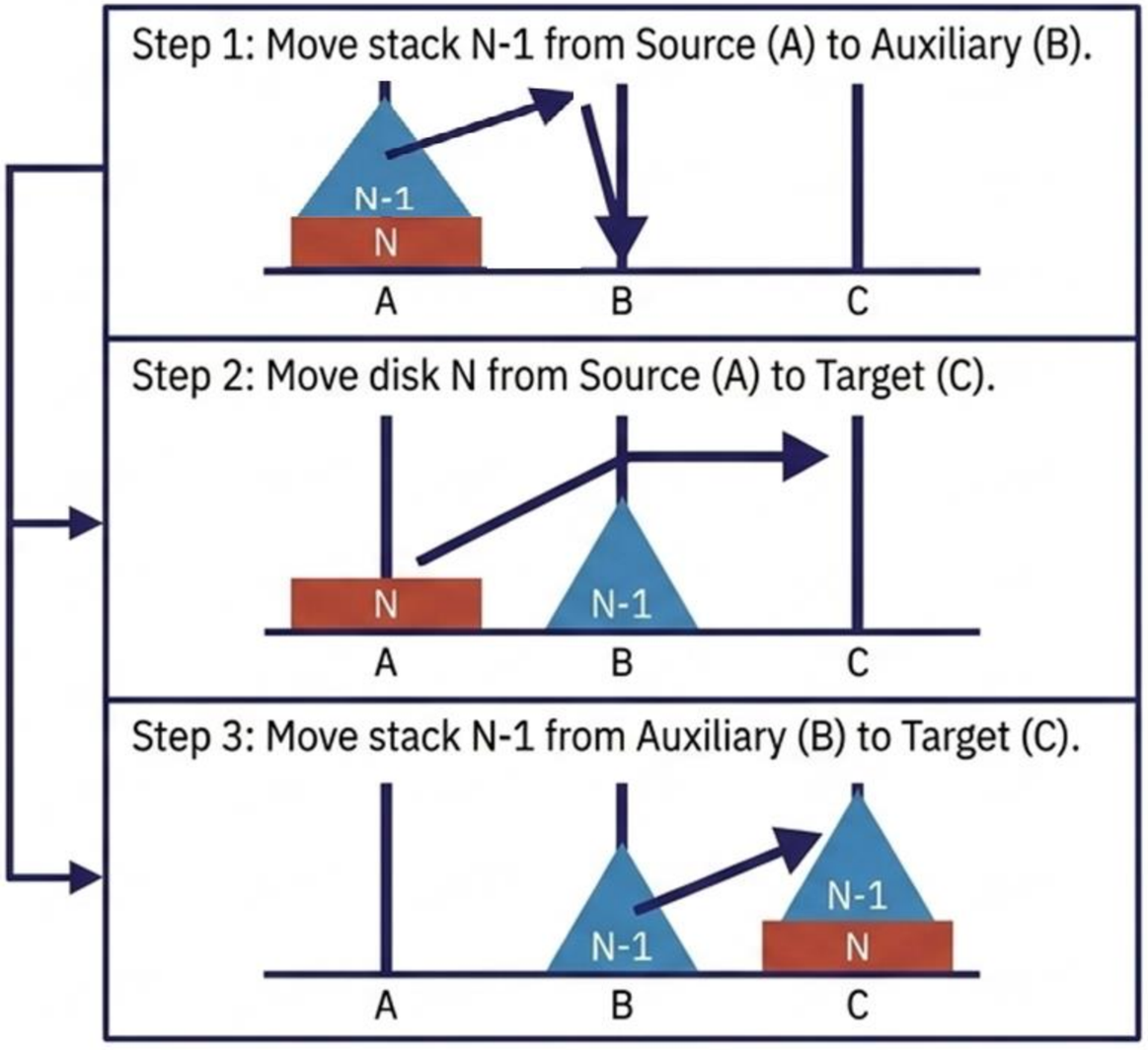


A larger disk can never be placed onto a smaller disk.

The 3-Disk Challenge: Physical Execution



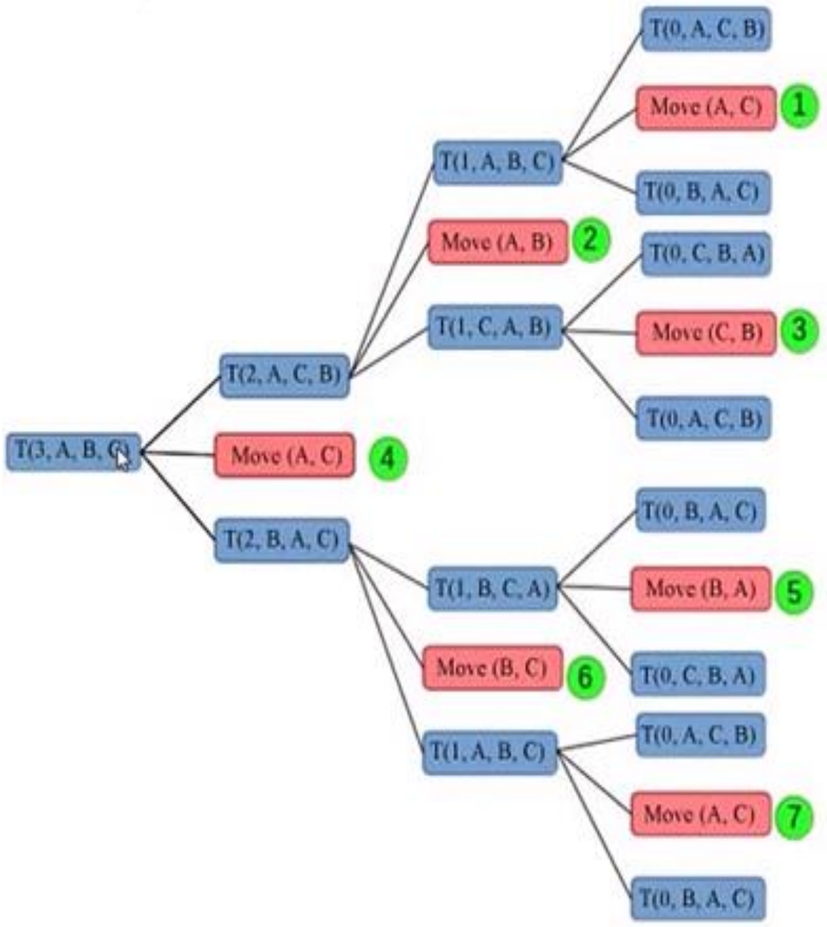
The Recursive Playbook

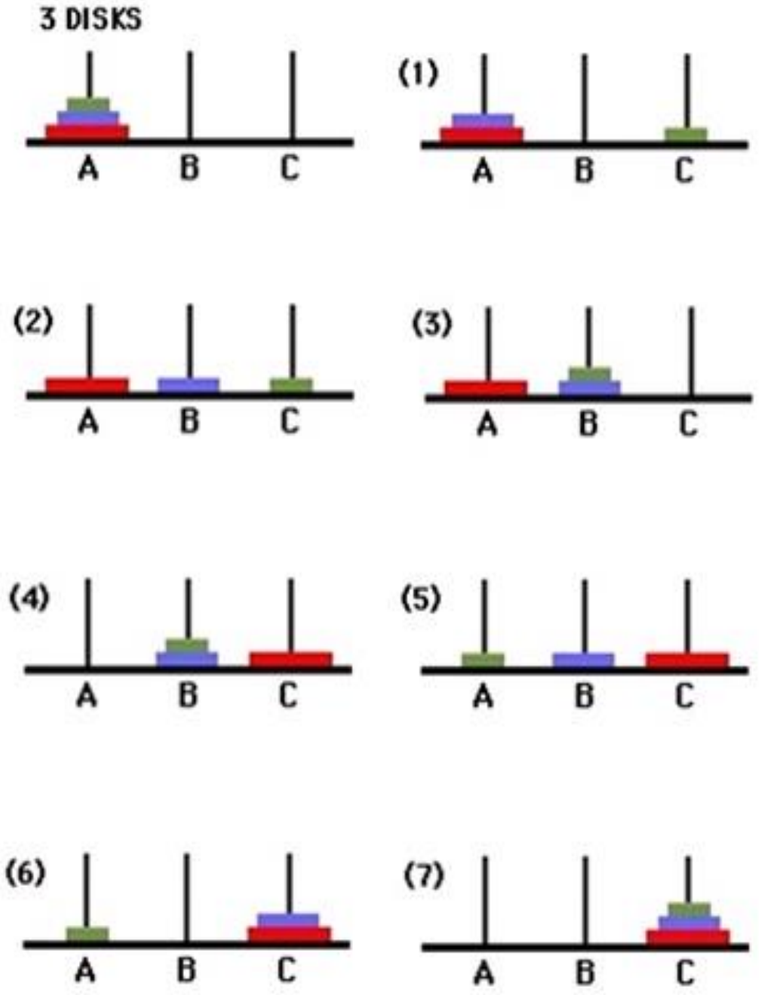


[Solve for N] = [Move N-1] + [Move N] + [Move N-1]

The Recursive Algorithm

```
tower(n, A, B, C) {
  if (n == 1)
    move A to C;
  else {
    tower(n-1, A, C, B);
    move A to C;
    tower(n-1, B, A, C);
  }
}
```





Four Methods for Solving Recurrences

1. Substitution (Guess & Prove)

Guessing the answer and proving its correctness using mathematical induction.

2. Iteration (Forward/Backward)

Repeatedly substituting the relation into itself until a solvable mathematical pattern emerges.

3. Recursion Tree

Visualizing recursive calls as a tree structure, calculating the cost at each level, and summing them up.

4. The Master Theorem

A direct formulaic framework providing immediate bounds for recurrences of the form $T(n) = aT(n/b) + f(n)$.

Method 2: Iteration applied to Tower of Hanoi

1. Pick recurrence and base case
2. Expand mathematically
3. Observe the pattern
4. Guess closed form
5. Prove via induction

$$T(n) = 2T(n-1) + 1$$

$$= 2(2T(n-2) + 1) + 1 = 2^2T(n-2) + 2 + 1$$

$$= 2^2(2T(n-3) + 1) + 2 + 1 = 2^3T(n-3) + 2^2 + 2 + 1$$

$$= \ldots$$

$$= 2^{n-1}T(n - (n-1)) + 2^{n-2} + \ldots + 2^2 + 2 + 1$$

Since $T(1) = 1$, sum is geometric series yielding $2^n - 1$

$$= 2^{n-1}T(n - (n-1)) + 2^{n-2} + \ldots + 2^2 + 2 + 1 = 2^n - 1$$

Final Complexity: $O(2^n)$

Method 2: Iteration (Linear Addition Example)

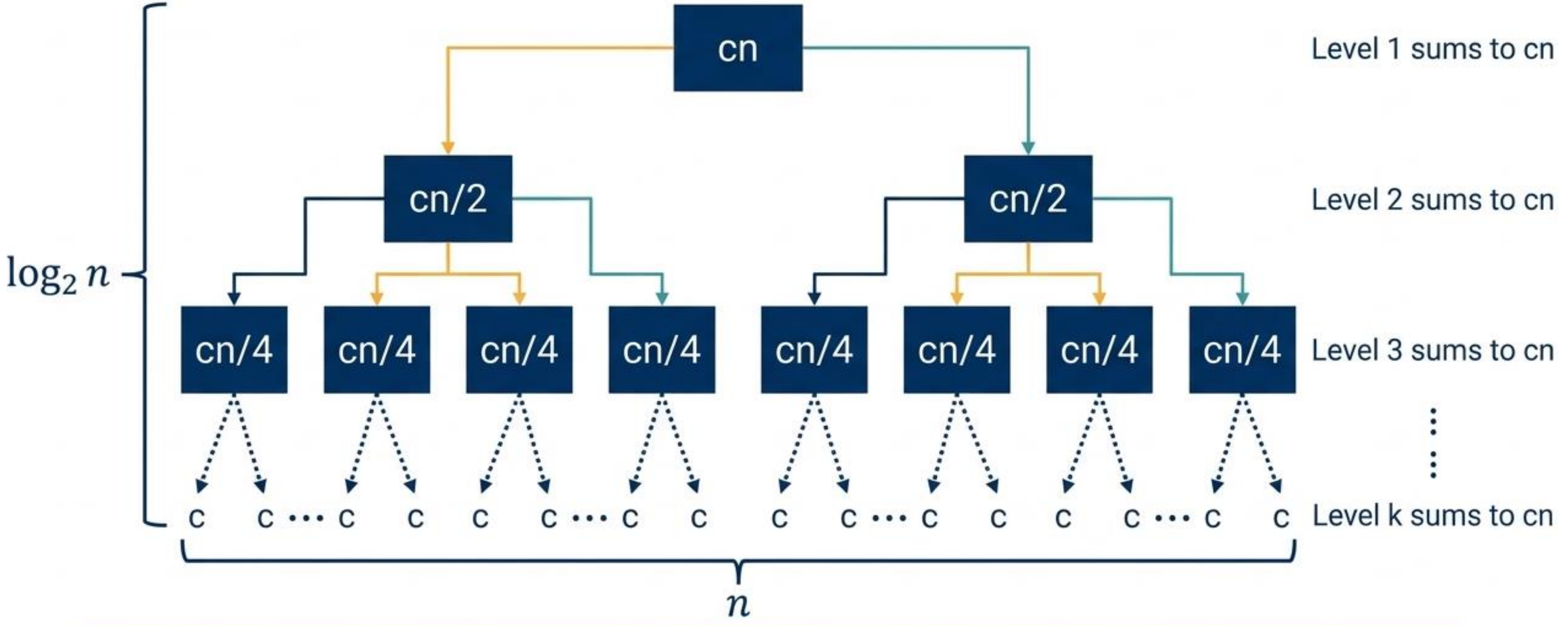
Given: $T(n) = n + T(n-1)$ and $T(1) = 1$

$$
\begin{aligned}
T(n) &= n + (n - 1) + T(n - 2) \\
&= n + (n - 1) + (n - 2) + T(n - 3) \\
&\quad \dots \\
&= n + (n - 1) + (n - 2) + \dots + 2 + T(1) \\
&= n + (n - 1) + (n - 2) + \dots + 2 + 1 \\
&= \text{Sum}(i=1 \text{ to } n) = n(n + 1) / 2
\end{aligned}
$$

Final Complexity: O(n^2)

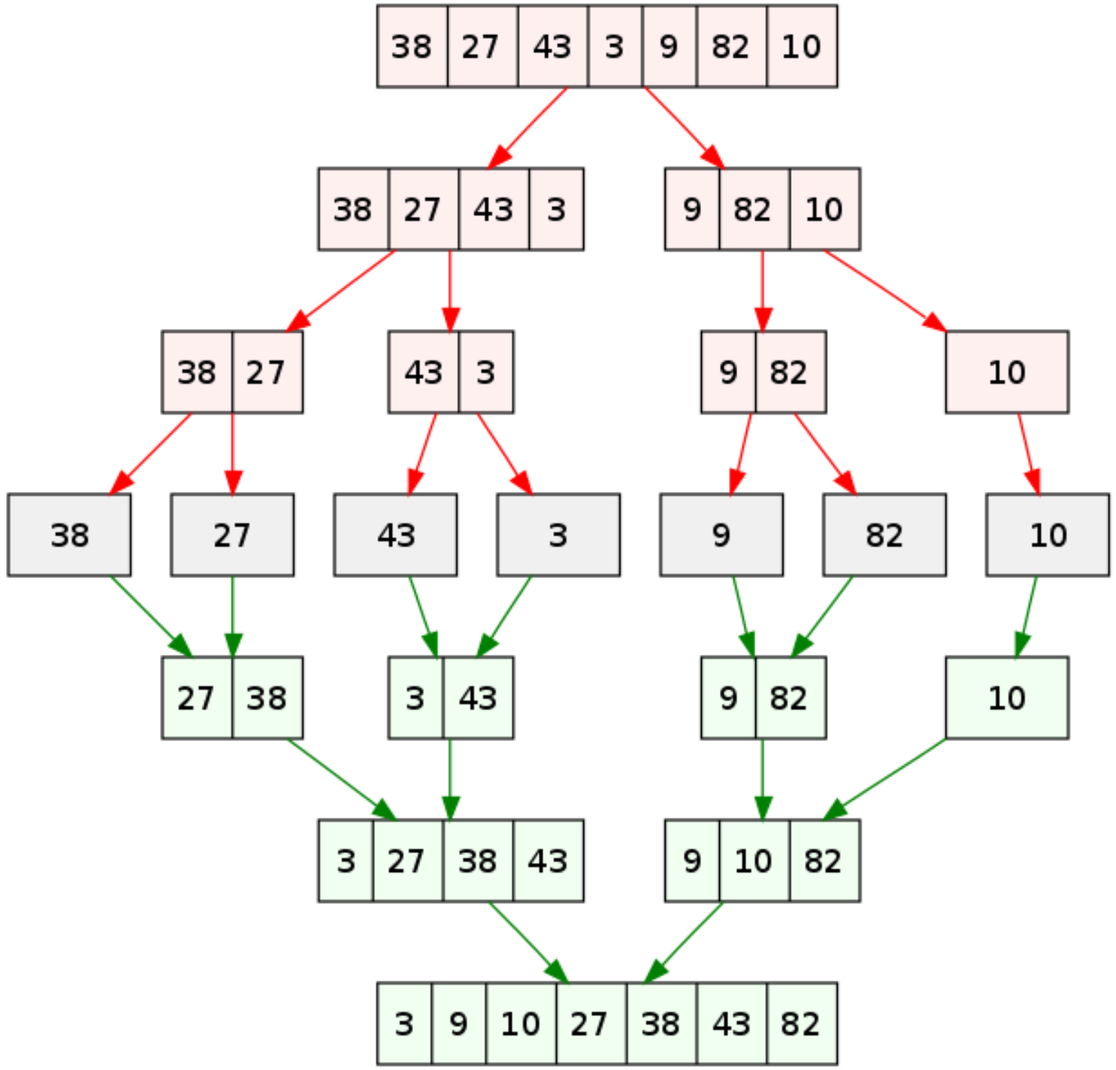
Method 3: Recursion Tree (Applied to Merge Sort)

$$T(n) = 2T(n/2) + cn$$



Total cost = (Cost per level × Height) + Leaf nodes = $cn \log n + cn$. **Final Complexity:** $O(n \log n)$

Merge Sort Clarification



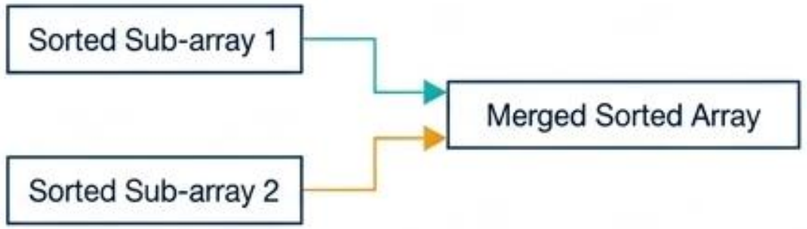
Algorithm Deep Dive: Merge Sort Structure

Main Algorithm Structure

```
MergeSort(Arr, left, right)
  if (left < right)
    mid = (left + right) / 2
    MergeSort(Arr, left, mid)      // T(n/2)
    MergeSort(Arr, mid+1, right)   // T(n/2)
    Merge(Arr, left, mid, right)   // Combine step, O(n)
```

The Merge() Helper

Takes two sorted sub-arrays and linearly traverses them to create one sorted array. This takes O(n) time, justifying the “cn” or “O(n)” term in the final recurrence relation.



Method 4: The Master Theorem (Diagnostic Framework)

Assume $a \geq 1$ and $b > 1$, for $T(n) = aT(n/b) + f(n)$

Case	Condition	Ultimate Complexity
Case 1	$f(n) < n^{\log_b a}$	$T(n) = \Theta(n^{\log_b a})$
Case 2	$f(n) == n^{\log_b a}$	$T(n) = \Theta(n^{\log_b a} * \log n)$
Case 3	$f(n) > n^{\log_b a}$	$T(n) = \Theta(f(n))$

Master Theorem Application (Cases 1 & 2)

Equation: $T(n) = 9T\left(\frac{n}{3}\right) + n$

Step 1: Calculate test factor:

$$n^{\log_b a} = n^{\log_3 9} = n^2.$$

Step 2: Compare: $f(n) = n$.

Since $n < n^2$, **Case 1** applies.

$$T(n) = \Theta(n^2)$$

Equation: $T(n) = 2T\left(\frac{n}{2}\right) + n$
(Merge Sort variant)

Step 1: Calculate test factor:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n.$$

Step 2: Compare: $f(n) = n$.

Since $n == n$, **Case 2** applies.

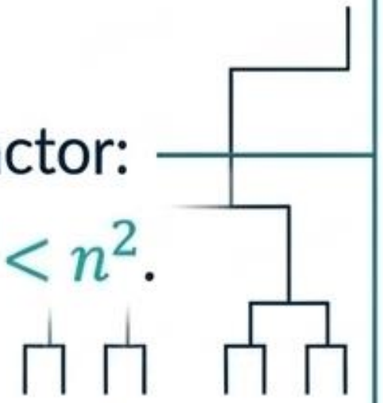
$$T(n) = \Theta(n \log n)$$

Master Theorem Application (Cases 1 & 3)

Equation: $T(n) = 3T\left(\frac{n}{4}\right) + n^2$

Step 1: Calculate test factor:

$$n^{\log_b a} = n^{\log_4 3} < n^2.$$



Step 2: Compare: $f(n) = n^2$.

Since $n^2 > n^{\log_4 3}$, **Case 3** applies.

$$T(n) = \Theta(n^2)$$

Equation: $T(n) = T\left(\frac{2n}{3}\right) + 1$
(Binary Search variant)

Step 1: Identify variables: a=1, b=3/2.

Step 2: Calculate test factor:

$$n^{\log_{3/2} 1} = n^0 = 1.$$

Step 3: Compare: $f(n) = 1$.

Since $1 == 1$, **Case 2** applies.

$$T(n) = \Theta(n \log n)$$

Master Theorem (Example 5)

Equation: $T(n) = 3T(n/4) + n \log n$

Step 1: Calculate test factor:

$n^{\log_4 3} = n^{0.793}$

Step 2: Compare: $f(n) = n \log n$

Since $n \log n > n^{0.793}$, Case 3 applies.

Conclusion: $T(n) = \text{Theta}(n \log n)$